TECHNISCHE
UNIVERSITÄT
MÜNCHEN

Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ARCHITECTURE FUNDAMENTALS

Reactive System Architecture

TECHNISCHE UNIVERSITÄT MÜNCHEN

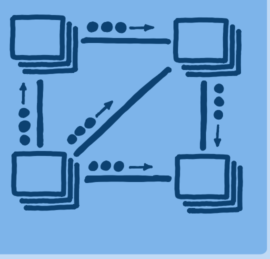
AF 10.4

Architecture & Systems

DEF Definition

Reactive System Architecture enables the realization of *Reactive Systems.*

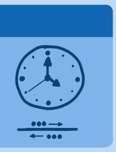
Reactive Systems are in *subordinated interaction* with their *dominating environment*. They *continuously* process *endless data streams* as *small messages*, react at *any time* and respond within *tight time limits*. For this, they continuously *observe* their *environment* and adapt their *behaviour* to the current situation.



Demand & Deliverables

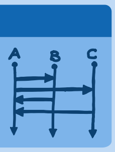
CTX Context

Real-time communication in the context of Digitization, Internet, Internet of Things (IoT), Systems of Engagement, Media and Analytics.



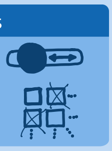
VAL Values

Non-blocking input/output data processing, fast responses within *tight time limits*, and *continuous availability* of the provided services.



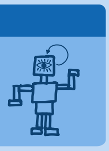
REQ Requirements

Services are *elastic* and provide high scalability, and are *resilient* and provide high fault tolerance.



PRP Properties

Services run *fully autonomously, monitor themselves,* and *automatically adapt* to changes in the environment.



Principles

Stay Responsive
Always respond in a timely manner.

Accept Uncertainty
Build reliability despite unreliable foundations.

Embrace Failure
Expect things to go wrong and design for resilience.

Assert Autonomy
Design components that act independently and interact collaboratively.

Tailor Consistency
Individualize consistency per component to balance availability and performance.

Decouple Time
Process asynchronously to avoid coordination and waiting.

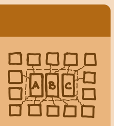
Decouple Space
Create flexibility by embracing the network.

Handle Dynamics
Continuously adapt to varying demand and resources.

Patterns & Paradigms

ARC Architecture

Microservices, Cloud-Native Architecture (CNA), Event-Driven Architecture (EDA).



+

EXE Execution

Parallelization, Concurrency, Actors, Threads, Thread-Pool, Event-Loop.

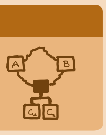
COM Communication

Asynchronous Communication, Non-Blocking I/O, Sequence, Push, Backpressure, Quality of Service (QoS).

+

INF Infrastructure

Message Queue (MQ), Load Balancer, Reverse Proxy, Service Mesh, Virtual Private Network (VPN).



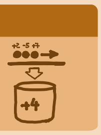
DAT Data

Semantical Event, Small Message, Endless Stream.

+

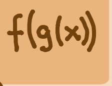
PRC Processing

Complex Event Processing (CEP), EAI Patterns, Stream Processing (map, flatMap, filter, reduce), Event Sourcing.



STY Style

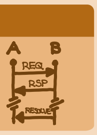
Functional Programming, Asynchronous Programming.



+

ASY Asynchronism

Callback, Promise/Future, Observable, Publish & Subscribe.



Reactive System Architecture enables the realization of **Reactive Systems**. Reactive Systems are in **subordinated interaction** with their **dominating environment**. They **continuously process endless data streams** as **small messages**, react at **any time** and respond within **tight time limits**. For this, they continuously **observe** their **environment** and **adapt** their **behaviour** to the current situation.

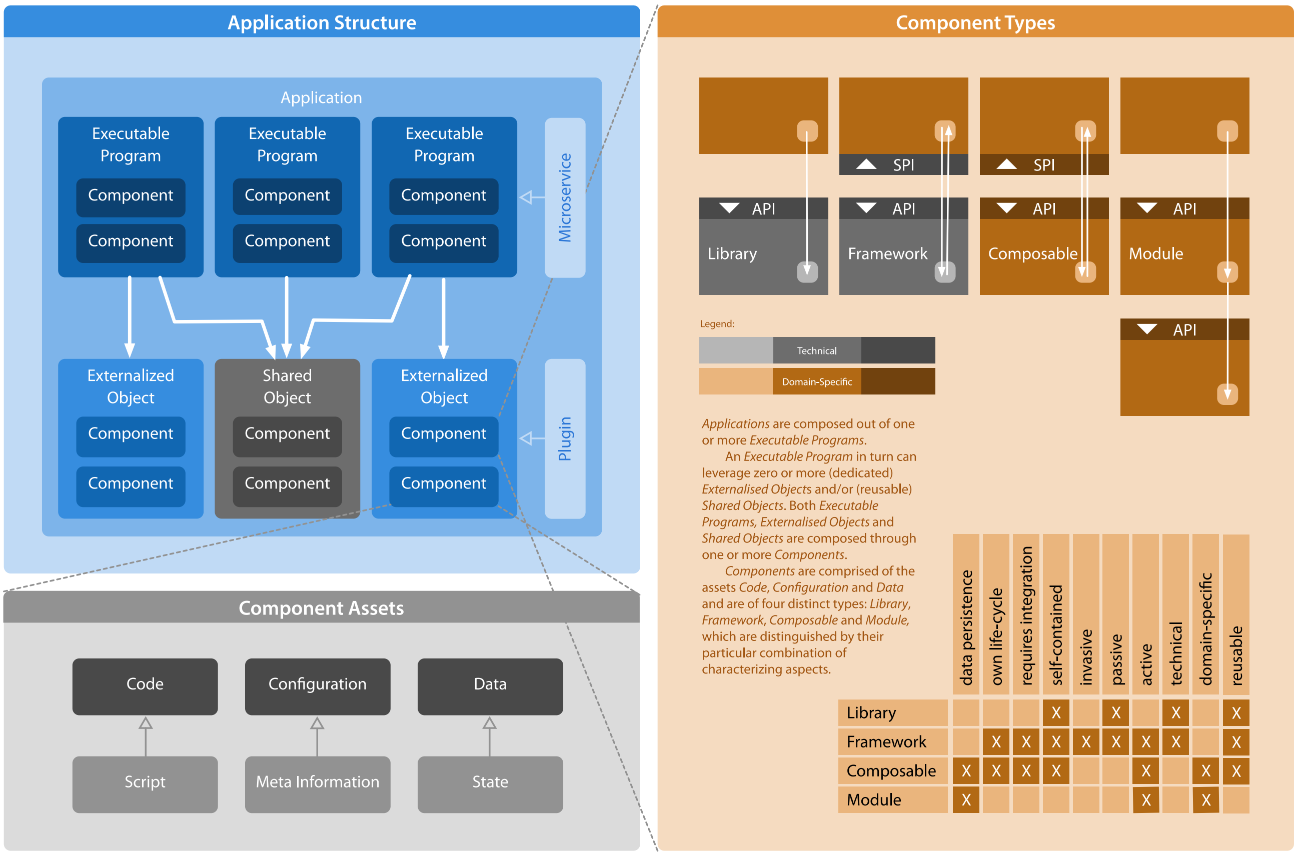
Reactive Systems are primarily used in the context of real-time communication where services are provided, which have to be **elastic** and provide high scalability, and which have to be **reliable** and provide high fault tolerance.

Questions

- ? Which two essential requirements do Reactive Systems fulfill?
- ? What characterizes Reactive Systems with respect to their data processing?

Intellectual Content: Version 1.1.1 (2020-11-09), Authored 2019-2020 by Michael Schäfer and Dr. Ralf S. Engelschall at msg Applied Technology Research
Graphical Illustration: Version 1.1.0 (2020-09-26), Copyright © 2019-2020 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.

Application Composition



Applications are composed out of one or more *Executable Programs*.

An *Executable Program* in turn can leverage zero or more (dedicated) *Externalised Objects* and/or (reusable) *Shared Objects*. Both *Executable Programs, Externalised Objects* and *Shared Objects* are composed through one or more *Components*.

Components are comprised of the assets *Code, Configuration* and *Data* and are of four distinct types: *Library, Framework, Composable* and *Module,* which are distinguished by their particular combination of characterizing aspects.

	data persistence	own life-cycle	requires integration	self-contained	invasive	passive	active	technical	domain-specific	reusable
Library				X		X		X		X
Framework		X	X	X	X	X	X	X		X
Composable	X	X	X	X			X		X	X
Module	X						X		X	

Applications are composed out of one or more Executable Programs. An Executable Program in turn can leverage zero or more (dedicated) Externalised Objects and/or (reusable) Shared Objects. Both Executable Programs, Externalised Objects and Shared Objects are composed of one or more Components. In a Microservice Architecture, the Executable Programs are called Microservices. In a Plugin Architecture, the Externalised Objects are called Plugins.

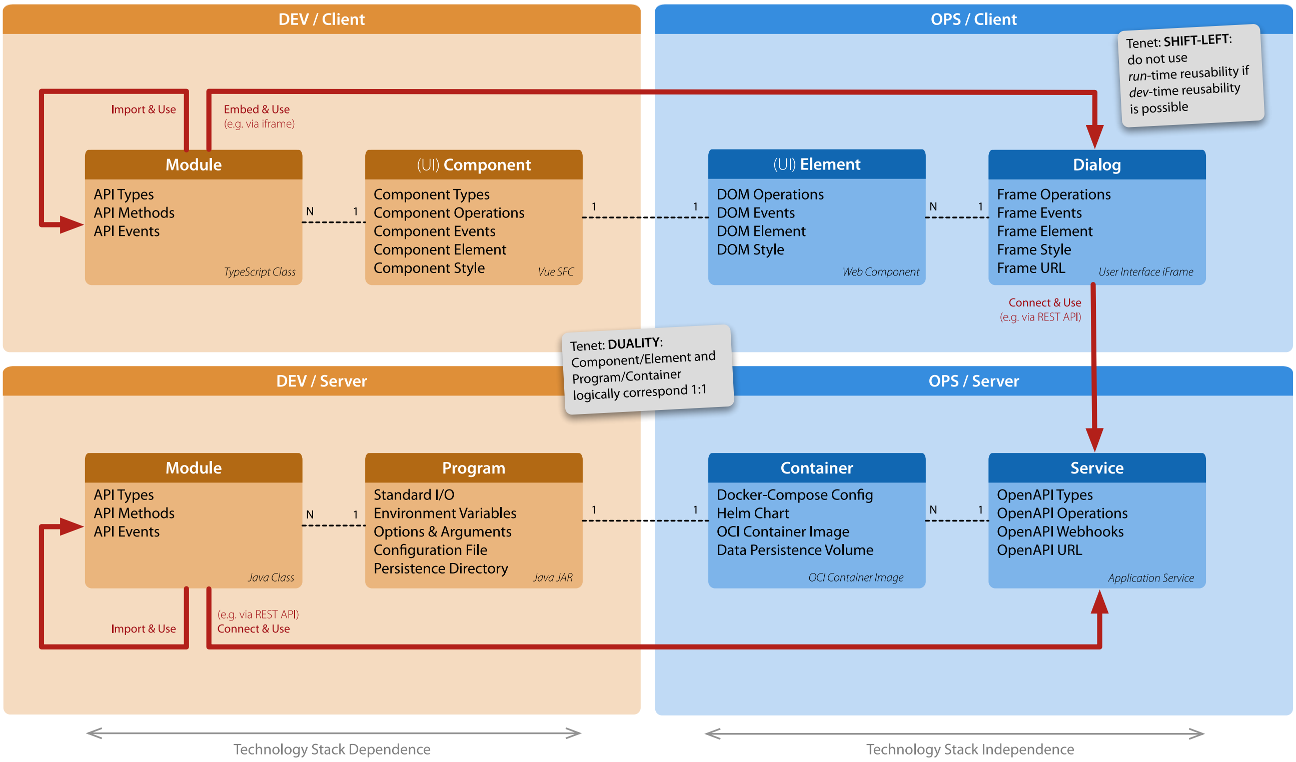
There are four distinct types of Components: Library, Framework, Composable and Module. They can be distinguished by their particular combination of characterizing aspects. Most prominently, whether they provide an Application Programming Interface (API) to the consumer of the Component and/or whether they require the consumer of the Component to fulfill some sort of Service Provider Interface (SPI).

Questions

- ? What is the main difference between a Library and a Framework?

Functional Reusability

Intellectual Content: Version 1.1.0 (2025-09-15), Authored 2024-2025 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.1.0 (2025-09-15), Copyright © 2024-2025 Dr. Ralf S. Engelschall <http://engelschall.com>, All Rights Reserved.



DEV / Client

OPS / Client

Tenet: **SHIFT-LEFT**: do not use *run*-time reusability if *dev*-time reusability is possible

Import & Use

Embed & Use
(e.g. via iframe)

Module
API Types
API Methods
API Events
TypeScript Class

(UI) Component
Component Types
Component Operations
Component Events
Component Element
Component Style
Vue SFC

(UI) Element
DOM Operations
DOM Events
DOM Element
DOM Style
Web Component

Dialog
Frame Operations
Frame Events
Frame Element
Frame Style
Frame URL
User Interface iFrame

Connect & Use
(e.g. via REST API)

Tenet: **DUALITY**: Component/Element and Program/Container logically correspond 1:1

DEV / Server

OPS / Server

Module
API Types
API Methods
API Events
Java Class

Program
Standard I/O
Environment Variables
Options & Arguments
Configuration File
Persistence Directory
Java JAR

Container
Docker-Compose Config
Helm Chart
OCI Container Image
Data Persistence Volume
OCI Container Image

Service
OpenAPI Types
OpenAPI Operations
OpenAPI Webhooks
OpenAPI URL
Application Service

Import & Use

(e.g. via REST API)
Connect & Use

Technology Stack Dependence

Technology Stack Independence

On the Client-side we distinguish the functional units **Module** and **Component** during Development (DEV) time, and **Element** and **Dialog** during Operations (OPS) time. The **Module** consists of the regular API Types, Methods and Events. The **Component** consists of the Web technology based Component Types (JS), Operations (JS), Events (DOM), Element (HTML) and Style (CSS). The **Element** consists of the Web technology based DOM Operations (JS), Events (DOM), Element (HTML) and Style (CSS). The **Dialog** is a User Interface Frame with Operations (JS), Events (DOM), Element (HTML), Style (CSS), and URL.

On the Client-side you can reuse functionality in a **Module** by either importing another **Module** (DEV), or embedding the Frame of an entire **Dialog** (OPS).

On the Server-side we distinguish the functional units **Module** and **Program** during Development (DEV) time, and **Container** and **Service** during Operations (OPS) time. The **Module** consists of the regular API Types, Methods and Events. The **Program** consists of Standard I/O, Environment Variables, Options & Arguments, Configuration File, and Persistence Directory. The **Container** consists of Docker-Compose Config and/or Helm Chart, OCI Container Image, and Data Persistence Volume. The **Service** consists of OpenAPI Types, Operations, Webhooks, and URL.

On the Server-side you can reuse functionality in a **Module** by either importing another **Module** (DEV), or connecting to a **Service** (OPS).

The Client-side **Dialog** also (re)uses the functionality of the Server-side **Service** by connecting to it.

Questions

? How can you **use** and **reuse** functionality in a Client/Server architecture?

Formal Languages

Declarative Languages

Express the target state
and let the machine figure out the steps.

Markup Languages

Write text intermixed with markup information.

```
foo <em>bar <strong>baz
</strong></em> quux
```

Examples:
Wiki, **Markdown**, AsciiDoc, SGML, **HTML**, TeX, R[un]off, reStructuredText, RTF

Configuration Languages

Express complex textual configurations.

```
foo bar quux { baz;
quux id 7; baz }
```

Examples:
INI, **XML**, SXML, **JSON**, **YAML**, TOML, HCL

Rule Languages

Express logic and semantic through complex rules.

```
foo(x, y) <- bar(x, y, z)
AND x < 42 AND z >= 10
```

Examples:
SQRL, Datalog/RuleML, OWL/**SWRL**, RIF

Constraint Languages

Find solutions for complex constraints.

```
foo @ bar(X, Y),
baz(X, Y, _) ==> quux.
```

Examples:
MiniZinc, CHR, **OCL**, Rego, Z3.

Query Languages

Retrieve information through paths and expressions.

```
// foo / bar [ @baz ==
"xxx" && @quux > 10 ]
```

Examples:
Glob, **RegExp**, **CSS Selector**, **XPath**, YARA, **GraphQL**, **SQL**, SPARQL, Cypher, GQL, **ASTq**

Validation Languages

Parse and validate complex textual information.

```
foo ::= “bar(#” (?:
[0-9a-fA-F]{2})+ “)”
```

Examples:
RegExp, Ducky, **BNF**, **PEG**, RELAX NG

solution approach: automatically, non-obvious
execution control: automatically, pre-defined
performance optimization: automatically, pre-defined

Imperative Languages

Express the steps
how the machine has to reach the target state.

Shell Languages

Automate execution of system commands.

```
foo -x 2>&1 | bar -y
--quux <(cat *.cf)
```

Examples:
Korn-Shell, Bourne-Shell, **Bash**, C-Shell, Batch-Script, **PowerShell**, AppleScript, DCL

Programming Languages

Execute complex algorithmic steps.

```
for (let i = 0; i < 10;
i++) foo(i, 42)
```

Examples:
JavaScript, **TypeScript**, **Scala**, **Kotlin**, Java, C#, C/C++, **Rust**, **Go**, Python, Perl, Ruby, Lua

Text-Processing Languages

Manipulate texts through transformations.

```
/^foo/,/bar.*baz/
s/quux\([0-9]*\)/foo\1/g
```

Examples:
ed, ex, **sed**, AWK, TXR, XSLT, JSLT

Macro Languages

Pre-process texts with macros.

```
define(`foo',`bar$1baz')
foo(quux)bar
```

Examples:
m4, GPP, CPP, **Zoem**, ProMac

Expression Languages

Expand path, arithmetic, and boolean expressions.

```
{{ foo.bar[*].baz[42]
.quux + 1 }}
```

Examples:
JQ, **YQ**, **MozJEXL**, MathML, JUEL, SpEL

Template Languages

Expand complex text fragments.

```
{% for k, v in items %}
{{k}}: {{v}}{% endfor %}
```

Examples:
Pug, **Nunjucks**, Handlebars, Mustache, Jinja, **Jsonnet**

solution approach: manually, obvious
execution control: manually, fine-grained
performance optimization: manually, fine-grained

Examples:
essential
recommended
alternative

There are innumerable **Formal Languages**. In a technology stack, usually, almost a dozen such languages are used at the same time. The architect must therefore select them very carefully.

The formal languages can first be divided into **Declarative Languages** and **Imperative Languages**. The former express a target state (“WHAT”); the latter express the way to get there (“HOW”).

Declarative approaches are usually to be preferred to imperative approaches because they leave it to the implementation (and not to the programmer) to find the optimal way. In addition, they permit incremental approaches, where the next step is determined by the particular difference between the current state and the desired target state. This is especially important for recovery in very dynamic and error-prone environments.

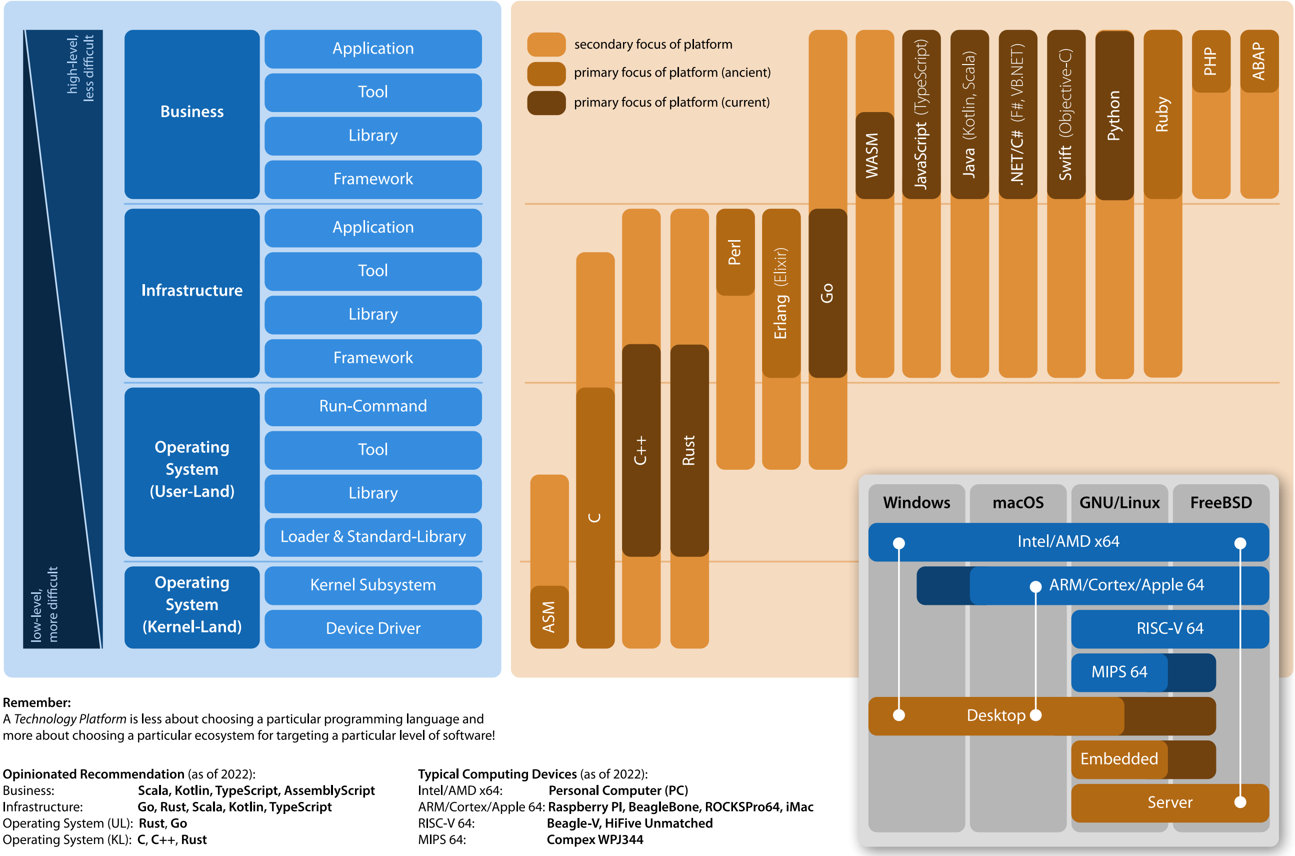
Questions

- Into which two classes can **Formal Languages** be divided?

ARCHITECTURE FUNDAMENTALS

Technology Platforms

TECHNISCHE UNIVERSITÄT MÜNCHEN



Remember:
A *Technology Platform* is less about choosing a particular programming language and more about choosing a particular ecosystem for targeting a particular level of software!

Opinionated Recommendation (as of 2022):

Business:	**Scala, Kotlin, TypeScript, AssemblyScript**
Infrastructure:	**Go, Rust, Scala, Kotlin, TypeScript**
Operating System (UL):	**Rust, Go**
Operating System (KL):	**C, C++, Rust**

Typical Computing Devices (as of 2022):

Intel/AMD x64:	**Personal Computer (PC)**
ARM/Cortex/Apple 64:	**Raspberry PI, BeagleBone, ROCKSPro64, iMac**
RISC-V 64:	**Beagle-V, HiFive Unmatched**
MIPS 64:	**Compex WPJ344**

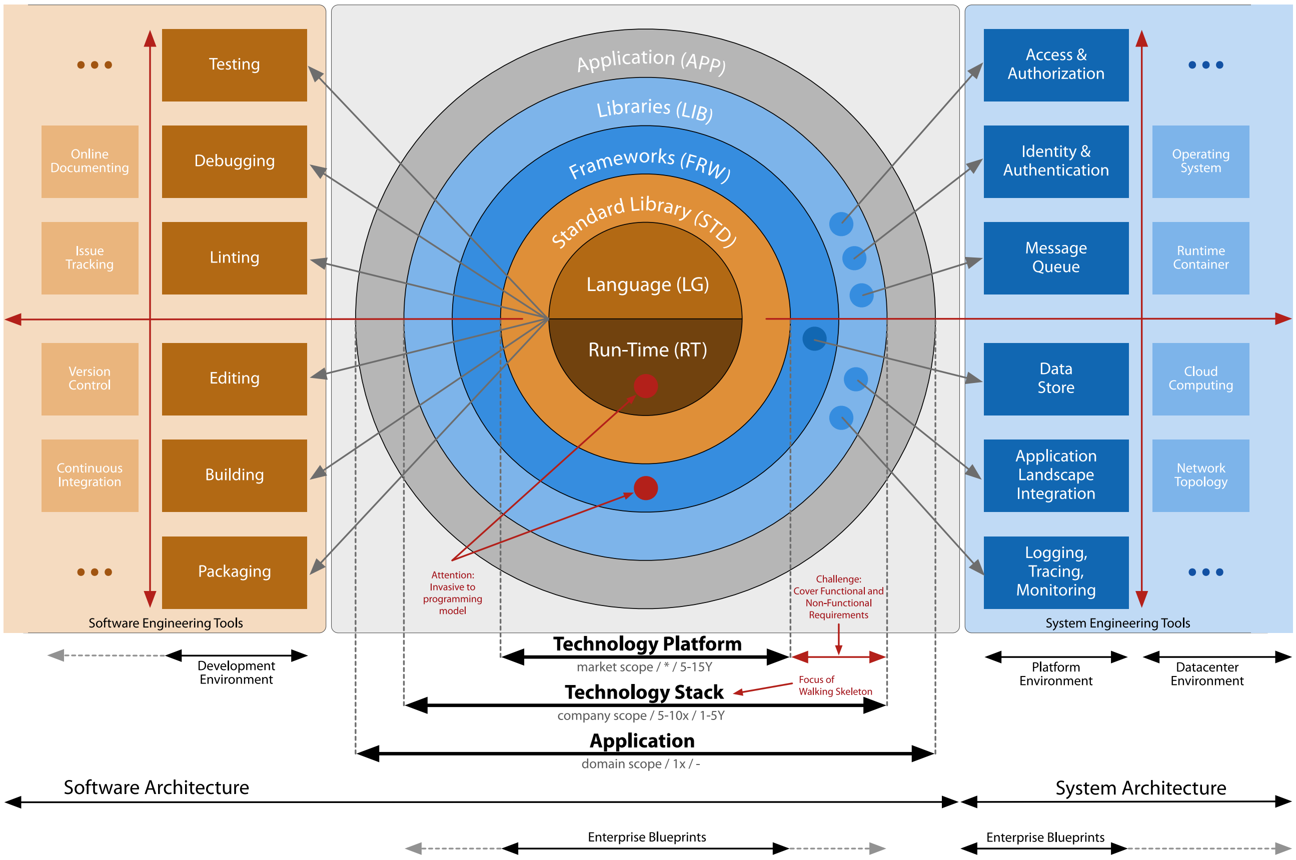
There are various levels of software, ranging from low-level and more difficult (operating system related) to high-level and less difficult (business-logic related).

A Technology Platform is less about choosing a particular programming language and more about choosing a particular ecosystem for targeting a particular kind of software!

Questions

- Is the Technology-Platform **Node.js** suitable to implement a Kernel-Subsystem at the level of the operating system?

Technology Stack



A **Technology Platform** consists of a **Language**, an optional **Run-Time** environment and a **Standard Library**. On top of this, **Frameworks** and **Libraries** extend this to a **Technology Stack**, in which with them especially all the prerequisites for the functional and non-functional requirements in the **Application** are achieved.

It has to be noted that the **Run-Time** and the **Frameworks** are usually extremely “invasive” to the programming model and thus can almost never be replaced afterwards. Therefore, with the so-called **Walking Skeleton**, the focus is mainly on the **Technology Stack** to be defined and integrated.

While the **Application** has a functional scope and is implemented only once, a particular **Technology Stack** is usually defined by a company and then reused several times over a period of a few years.

The underlying particular **Technology Platform**, on the other hand, is implemented by a third party for the market, is reused as often as required and must exist for quite a long period of time.

Large companies, therefore, usually stringently define the **Technology Platforms** and **Technology Stacks** in their **Enterprise Blueprints**.

For the **Software Engineering Tools** one should take into account the tools for **Testing**, **Debugging**, **Linting**, **Editing**, **Building** and **Packaging** of the **Development Environment**, because these are usually directly dependent on the particular **Technology Stack**.

The situation is similar for the **System Engineering Tools** of the **Platform Environment**: these require at least the associated **Libraries** in the **Technology Stack**, in order to be addressed during the run-time of the **Application**.

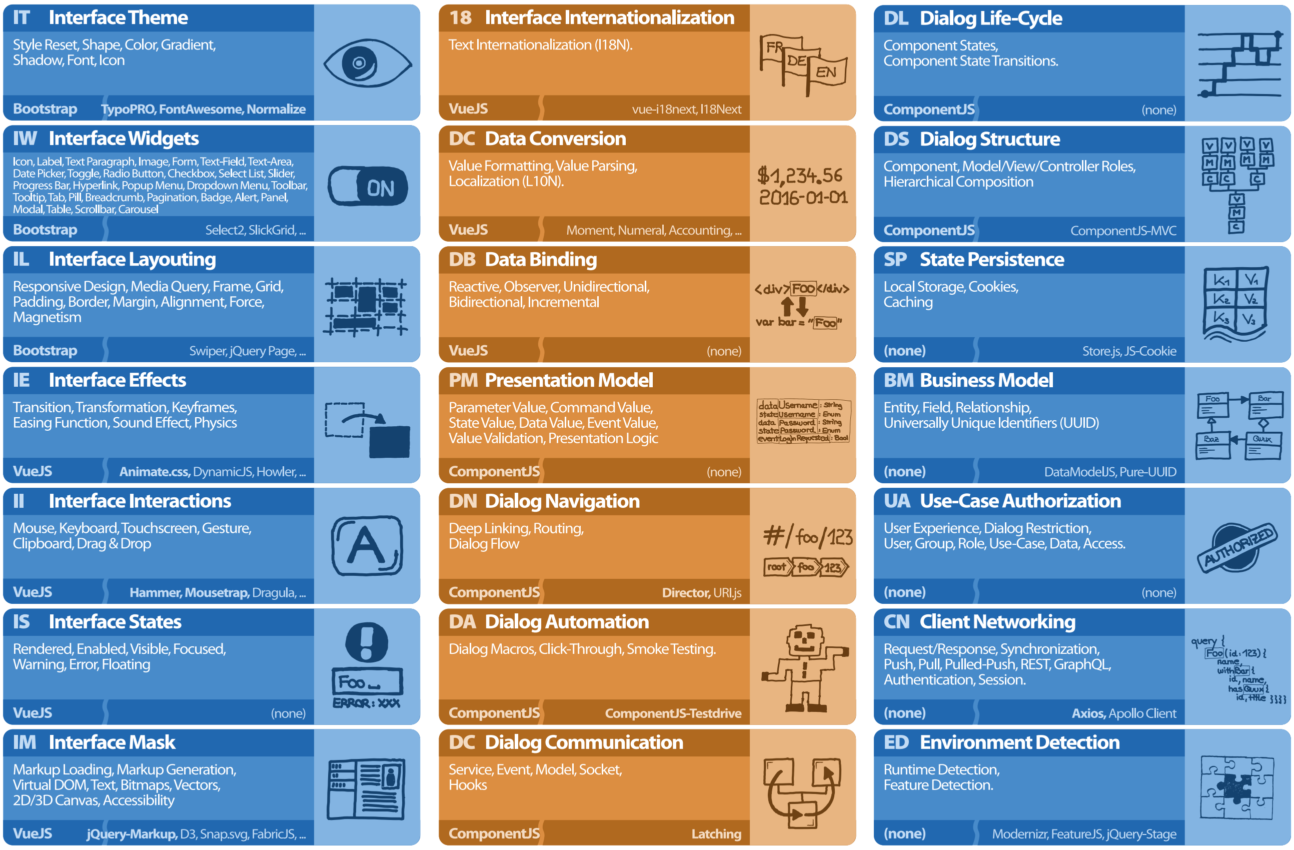
Questions

- What are the three components of a **Technology Platform**?
- Which two additional components make up a **Technology Stack**, compared to the **Technology Platform**?
- Which two components of a **Technology Stack** are most “invasive” to the programming model?

ARCHITECTURE
FUNDAMENTALS

Rich-Client Aspects

TECHNISCHE
UNIVERSITÄT
MÜNCHEN



IT	**Interface Theme**	Style Reset, Shape, Color, Gradient, Shadow, Font, Icon	**Bootstrap**	**TypoPRO, FontAwesome, Normalize**
IW	**Interface Widgets**	Icon, Label, Text Paragraph, Image, Form, Text-Field, Text-Area, Date Picker, Toggle, Radio Button, Checkbox, Select List, Slider, Progress Bar, Hyperlink, Popup Menu, Dropdown Menu, Toolbar, Tooltip, Tab, Pill, Breadcrumb, Pagination, Badge, Alert, Panel, Modal, Table, Scrollbar, Carousel	**Bootstrap**	Select2, SlickGrid, ...
IL	**Interface Layouting**	Responsive Design, Media Query, Frame, Grid, Padding, Border, Margin, Alignment, Force, Magnetism	**Bootstrap**	Swiper, jQuery Page, ...
IE	**Interface Effects**	Transition, Transformation, Keyframes, Easing Function, Sound Effect, Physics	**VueJS**	**Animate.css,** DynamicJS, Howler, ...
II	**Interface Interactions**	Mouse, Keyboard, Touchscreen, Gesture, Clipboard, Drag & Drop	**VueJS**	**Hammer, Mousetrap,** Dragula, ...
IS	**Interface States**	Rendered, Enabled, Visible, Focused, Warning, Error, Floating	**VueJS**	(none)
IM	**Interface Mask**	Markup Loading, Markup Generation, Virtual DOM, Text, Bitmaps, Vectors, 2D/3D Canvas, Accessibility	**VueJS**	**jQuery-Markup,** D3, Snap.svg, FabricJS, ...
18	**Interface Internationalization**	Text Internationalization (I18N).	**VueJS**	vue-i18next, I18Next
DC	**Data Conversion**	Value Formatting, Value Parsing, Localization (L10N).	**VueJS**	Moment, Numeral, Accounting, ...
DB	**Data Binding**	Reactive, Observer, Unidirectional, Bidirectional, Incremental	**VueJS**	(none)
PM	**Presentation Model**	Parameter Value, Command Value, State Value, Data Value, Event Value, Value Validation, Presentation Logic	**ComponentJS**	(none)
DN	**Dialog Navigation**	Deep Linking, Routing, Dialog Flow	**ComponentJS**	**Director,** URI.js
DA	**Dialog Automation**	Dialog Macros, Click-Through, Smoke Testing.	**ComponentJS**	**ComponentJS-Testdrive**
DC	**Dialog Communication**	Service, Event, Model, Socket, Hooks	**ComponentJS**	**Latching**
DL	**Dialog Life-Cycle**	Component States, Component State Transitions.	**ComponentJS**	(none)
DS	**Dialog Structure**	Component, Model/View/Controller Roles, Hierarchical Composition	**ComponentJS**	ComponentJS-MVC
SP	**State Persistence**	Local Storage, Cookies, Caching	**(none)**	Store.js, JS-Cookie
BM	**Business Model**	Entity, Field, Relationship, Universally Unique Identifiers (UUID)	**(none)**	DataModelJS, Pure-UUID
UA	**Use-Case Authorization**	User Experience, Dialog Restriction, User, Group, Role, Use-Case, Data, Access.	**(none)**	(none)
CN	**Client Networking**	Request/Response, Synchronization, Push, Pull, Pulled-Push, REST, GraphQL, Authentication, Session.	**(none)**	**Axios,** Apollo Client
ED	**Environment Detection**	Runtime Detection, Feature Detection.	**(none)**	Modernizr, FeatureJS, jQuery-Stage

To define a Technology Stack for a **Rich Client**, 21 **Aspects** have to be considered. Each aspect is covered by at least one **Framework** or **Library**. In practice, each aspect is usually covered by one Framework and zero or more Libraries. The goal always is: to achieve the greatest possible coverage of the aspects with a minimum number of Frameworks and Libraries.

It is advisable to use Open Source Software (OSS) for both Frameworks and Libraries and, if possible, to use no custom implementations, as the effort usually is not in proportion to the benefit, because all aspects are technical — and not functional — aspects of a user interface.

In the case of a Thin-Client Architecture (instead of a Rich-Client Architecture), a few aspects like **Client Networking** and **Environment Detection** are omitted. All other aspects are still valid, even if, in the case of a Thin-Client Architecture, the frontend (and thus the aspects of the user interface) of the application runs on the server.

Two important aspects deal with the data model: the **Business Model** is a data model that comes directly from the server and is exactly the same in slicing and granularity as the business data model of the server. Its data is synchronized with a **Presentation Model**, which in slicing and granularity is more like the (more technical) data model of the user interface (especially via the aspects **Interface Mask** and **Data Binding**).

Questions

- ? How does one ensure in a **Rich-Client Architecture** that the numerous technical **Aspects** of a user interface are addressed?

- ? Which two **Aspects** of a **Rich-Client Architecture** hold the data model and take care of the fact that the data supplied by the server cannot be used directly in the user interface?

ARCHITECTURE
FUNDAMENTALS

Thin-Server Aspects

TECHNISCHE
UNIVERSITÄT
MÜNCHEN

ED	**Environment Detection**	Detect the run-time environment, like underlying operating system, execution platform, network topology, feature toggles, etc.	Node	process, syspath
AP	**Argument Parsing**	Parse options and arguments of the Command-Line Interface (CLI) to bootstrap application parameters.	(none)	yargs
CP	**Configuration Parsing**	Load and parse directives from configuration file to bootstrap application parameters.	(none)	js-YAML
PD	**Process Daemonizing**	Detach from the startup terminal and host process in order to run fully independently.	(none)	daemonize2
PM	**Process Management**	(Pre-)fork child processes and/or threads of execution and monitor and control them during the life-cycle of the application.	(none)	cluster, nodemon
CM	**Component Management**	Structure the code into components, instanciate them under run-time and manage them in a stateful component life-cycle.	Microkernel	(none)
CC	**Component Communication**	Provide inter-component communication mechanisms like events, hooks, registry, etc.	Microkernel	Latching
SN	**Server Networking**	Listen to network sockets, accept connections and manage request/response and message communication.	HAPI	hapi-plugin-websocket, ws
PI	**Peer Information**	Determine unique identification and add-on information about the client peer.	HAPI	hapi-plugin-peer, geoip
SH	**Session Handling**	Manage secured per-connection sessions to keep state between communication requests and/or client sessions.	HAPI	YAR
UA	**User Authentication**	Determine and validate the unique identity of the user communicating over the current network connection.	HAPI	JWT, Passport
RV	**Request Validation**	Validate the syntactical and semantical compliance of the requests and sanitize the requests.	HAPI	Joi, DuckyJS
RP	**Request Processing**	Process the request by dispatching execution according to the provided request and determined context information.	HAPI	GraphQL.js
RA	**Role Authorization**	Determine whether the role of the current user is allowed to execute the current request.	(none)	GraphQL-Tools-Sequelize
CN	**Client Networking**	Provide mechanisms to connect to peers over the network and perform request/response and/or publish/subscribe communication.	(none)	Axios, MQTT.js, ws
TS	**Task Scheduling**	Schedule and execute recurring tasks independent of regular I/O operations.	(none)	node-scheduler
ET	**Execution Tracing**	Provide mechanisms for tracing the execution by logging event and measurement information at certain points of interest.	Microkernel	Winston
DA	**Database Access**	Map in-memory domain entities onto data store dependent persistent data structure.	Sequelize	GraphQL-Tools-Sequelize
DC	**Database Connectivity**	Locally or remotely connect the database access layer to the underlying data store.	Sequelize	sqlite3, pg
DS	**Database Schema**	Create, update or downgrade the data schema inside the underlying data store.	Sequelize	(none)
DB	**Database Bootstrapping**	Create, update or downgrade both mandatory bootstrapping and optional domain-specific data inside the underlying data store.	Sequelize	ini

To define a Technology Stack for a **(Thin-)Server**, 21 **Aspects** have to be considered. Each Aspect is covered by at least one **Framework** or **Library**. In practice, each Aspect is usually covered by one Framework and zero or more Libraries. The goal always is: to achieve the greatest possible coverage of the Aspects with a minimum number of Frameworks and Libraries.

It is advisable to use Open Source Software (OSS) for both Frameworks and Libraries and, if possible, to use no custom implementations, as the effort usually is not in proportion to the benefit, because all Aspects are technical — and not functional — Aspects of a user interface.

It is to be noted that a server usually does not only have the Aspect **Server Networking** (for the connection of the Rich Clients), but also the Aspect **Client Networking**, in order to be able to query other servers.

In addition, it is to be noted that, above all, two important Aspects address security issues: the Aspect **User Authentication** identifies and authenticates the user ("Is the user the one?"). The Aspect **Role Authorization**, on the other hand, before all business processes, checks whether the authenticated user really is authorized to initiate the processes due to his role(s) ("Is the user allowed to do this?").

Questions

- Why does a **(Thin-)Server** usually have, besides the obvious aspect **Server Networking**, also the aspect **Client Networking**?
- Which Aspect of a **(Thin-)Server** takes care of the question "Is the user the one"?
- Which aspect of a **(Thin-)Server** takes care of the question "Is the user allowed to do this"?